

# MANUAL DE GITHUB DE NOVATO A AVANZADO

---

Guía exhaustiva para desarrolladores en macOS con chips Apple Silicon (M1/M2/M3/M4), Zsh, Keychain y GitHub Actions

[CLI & Web Workflows](#)[SSH & GPG Signing](#)[GitHub Actions CI/CD](#)[Issues & Projects v2](#)[Security & Dependabot](#)[Packages & Releases](#)[API & Scripting](#)

# Manual de GitHub: De Novato a Avanzado en macOS (Apple Silicon M1/M2/M3/M4)

---

**Plataforma:** macOS Sonoma / Sequoia / versiones modernas con arquitectura ARM64 (Apple Silicon)

**Shell:** Zsh (Z Shell por defecto de macOS)

**Herramientas:** Xcode Command Line Tools, Homebrew en `/opt/homebrew`, GitHub CLI (`gh`), Apple Keychain (`osxkeychain`), OpenSSH con Touch ID / Secure Enclave, Git LFS

---

## Índice de Contenidos

1. **Parte I: Fundamentos y Configuración del Entorno en Apple Silicon**
  - 1.1 Diferencias entre Git y GitHub
  - 1.2 Xcode CLI Tools y Homebrew ARM64 en `/opt/homebrew`
  - 1.3 Configuración de Identidad y Limpieza Global de `.DS_Store`
  - 1.4 Integración con Apple Keychain (`osxkeychain`) y GitHub CLI
  - 1.5 Autenticación SSH Avanzada: Claves Ed25519 y Touch ID
  - 1.6 Firma Criptográfica de Commits con SSH y GPG
2. **Parte II: Flujo de Trabajo Esencial (Nivel Novato)**
  - 2.1 Creación y Clonación de Repositorios desde Terminal
  - 2.2 El Ciclo de Estados: Working Tree, Index y Commit en Zsh
  - 2.3 Staging Selectivo y Convención de Commits
  - 2.4 Sincronización con GitHub
  - 2.5 Control de Archivos Ignorados en macOS
3. **Parte III: Ramas, Fusiones y Estrategias Colaborativas (Nivel Intermedio)**
  - 3.1 Gestión Eficiente de Ramas con `git switch` y Zsh
  - 3.2 Los Tres Métodos de Merge en GitHub
  - 3.3 Modelos de Flujo: GitHub Flow, Git Flow y Forking
  - 3.4 Operaciones de Reorganización: Stash, Cherry-Pick y Rebase Interactivo
  - 3.5 El Salvavidas: Recuperación con Reflog
4. **Parte IV: Soluciones por Temas a la Edición Concurrente del Mismo Archivo**
  - 4.1 Tema 1: Prevención y Buenas Prácticas de Equipo
  - 4.2 Tema 2: Fusión Automática en Distintas Líneas
  - 4.3 Tema 3: Conflicto Directo de Fusión en macOS
  - 4.4 Tema 4: Elección Total de Versión

- 4.5 Tema 5: Cambios Locales sin Confirmar al hacer Pull
  - 4.6 Tema 6: Push Rechazado por Desfase y Rebase Seguro
  - 4.7 Tema 7: Resolución de Conflictos en Pull Requests
  - 4.8 Tema 8: Conflicto de Modificación vs Eliminación
  - 4.9 Tema 9: Archivos Binarios y Bloqueo con Git LFS en macOS
5. Parte V: Herramientas Modernas de Productividad Avanzada
- 5.1 Git Worktrees en macOS: Múltiples Ramas sin Conmutar
  - 5.2 Depuración Binaria con Git Bisect y Rastreo con Blame
  - 5.3 GitHub Codespaces desde la Terminal de Mac
  - 5.4 GitHub Copilot CLI en Zsh
  - 5.5 Git Hooks y Validación con Pre-commit en macOS
6. Parte VI: Gestión de Proyectos y Ecosistema GitHub
- 6.1 GitHub Issues, Hitos y Etiquetas desde Terminal
  - 6.2 Pull Requests y Revisiones de Código con GitHub CLI
  - 6.3 GitHub Projects (v2): Tableros y Automatización
  - 6.4 GitHub Discussions y Wikis Locales
7. Parte VII: Automatización y CI/CD con GitHub Actions
- 7.1 Estructura de Workflows y Sintaxis YAML
  - 7.2 Pipelines con Runners Apple Silicon ( `macos-14` , `macos-15` )
  - 7.3 Compilación Cruzada, Binarios Universales y Caching
  - 7.4 Configuración de un Self-Hosted Runner ARM64 como Launchd Daemon
8. Parte VIII: Distribución, Paquetes y Publicación
- 8.1 GitHub Releases: Binarios `.dmg` , `.pkg` y Arquitecturas ARM64
  - 8.2 GitHub Packages y Distribución mediante Homebrew Taps
  - 8.3 GitHub Pages: Sitios Estáticos y Documentación
9. Parte IX: Seguridad, Gobernanza y Políticas de Repositorio
- 9.1 Reglas de Protección de Ramas y Rulesets
  - 9.2 Dependabot, Secret Scanning y Push Protection
  - 9.3 Análisis Estático con CodeQL
  - 9.4 Gobernanza con `CODEOWNERS` y Permisos de Equipo
10. Parte X: Catálogo Maestro de Incidentes y Soluciones en macOS
- 10.1 Incidente 1: Fuga Accidental de Secretos en macOS
  - 10.2 Incidente 2: Push Rechazado por Archivo Mayor a 100 MB
  - 10.3 Incidente 3: Reversión Limpia de un Merge Roto en Producción

- 10.4 [Incidente 4: Rebase de una Rama Compartida](#)
  - 10.5 [Incidente 5: Resurrección de una Rama Remota Borrada en GitHub](#)
  - 10.6 [Incidente 6: Corrección Masiva de Correo en Commits Históricos](#)
  - 10.7 [Incidente 7: Ataques de Pwn Request en Runners de macOS](#)
  - 10.8 [Incidente 8: Bucle Infinito de Workflows en GitHub Actions](#)
  - 10.9 [Incidente 9: Optimización y Poda de Objetos Huérfanos en macOS](#)
  - 10.10 [Incidente 10: Tags o Etiquetas Desincronizadas en macOS](#)
-

# Parte I: Fundamentos y Configuración del Entorno en Apple Silicon

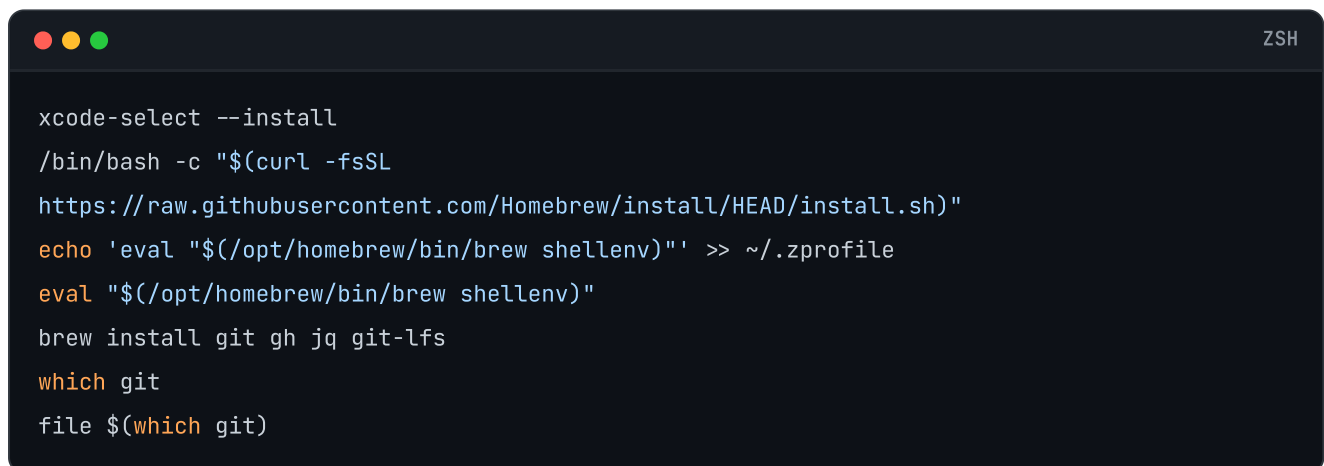
---

## 1.1 Diferencias entre Git y GitHub

- **Git:** Sistema de control de versiones descentralizado optimizado para computación local de alto rendimiento. En macOS con procesadores Apple Silicon (M1/M2/M3/M4), Git aprovecha la arquitectura ARM64 para operaciones de árbol y cómputo de hashes ultrarrápidas.
  - **GitHub:** Plataforma web y servicio de nube integral para la colaboración, revisión de código, flujos CI/CD con runners nativos de macOS, gobernanza corporativa y distribución de software.
- 

## 1.2 Xcode CLI Tools y Homebrew ARM64 en `/opt/homebrew`

En Apple Silicon, Homebrew se instala en `/opt/homebrew` (a diferencia de `/usr/local` de Intel), garantizando binarios nativos ARM64 sin emulación por Rosetta 2.



```

xcode-select --install
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile
eval "$(/opt/homebrew/bin/brew shellenv)"
brew install git gh jq git-lfs
which git
file $(which git)
```

### ¿Qué hace este comando?

Instala las herramientas de compilación de Apple, configura Homebrew nativo para procesadores M1-M4 en `/opt/homebrew`, instala las versiones más recientes de Git, GitHub CLI, `jq` y Git LFS, y comprueba que el ejecutable sea un binario nativo Mach-O ARM64.

---

## 1.3 Configuración de Identidad y Limpieza Global de `.DS_Store`

```
git config --global user.name "Tu Nombre Completo"
git config --global user.email "tu-correo@ejemplo.com"
git config --global core.autocrlf input
git config --global init.defaultBranch main

cat << 'EOF' > ~/.gitignore_global
.DS_Store
.AppleDouble
.LSOoverride
._*
.Spotlight-V100
.Trashes
EOF

git config --global core.excludesfile ~/.gitignore_global
```

### ¿Qué hace este comando?

Define la autoría de tus confirmaciones, normaliza finales de línea a Unix LF, establece `main` como rama inicial y vincula un archivo de exclusiones global para que macOS nunca suba archivos `.DS_Store` a ningún repositorio.

## 1.4 Integración con Apple Keychain (`osxkeychain`) y GitHub CLI

```
git config --global credential.helper osxkeychain
gh auth login --hostname github.com --git-protocol ssh --web
gh auth status
```

### ¿Qué hace este comando?

Activa el asistente oficial del Llavero de Apple (Apple Keychain), que almacena contraseñas cifradas en el Secure Enclave del Mac, e inicia sesión en GitHub CLI a través de la web.

## 1.5 Autenticación SSH Avanzada: Claves Ed25519 y Touch ID

### Clave Ed25519 con persistencia en el Keychain de macOS

```
ssh-keygen -t ed25519 -C "tu-correo@ejemplo.com" -f ~/.ssh/id_ed25519

cat << 'EOF' >> ~/.ssh/config
Host github.com
    HostName ssh.github.com
    Port 443
    User git
    IdentityFile ~/.ssh/id_ed25519
    AddKeysToAgent yes
    UseKeychain yes
EOF
chmod 600 ~/.ssh/config

ssh-add --apple-use-keychain ~/.ssh/id_ed25519
gh ssh-key add ~/.ssh/id_ed25519.pub --title "MacBook-AppleSilicon"
ssh -T git@github.com
```

#### ¿Qué hace este comando?

Genera la clave Ed25519, configura `UseKeychain yes` en el puerto 443 para eludir firewalls, guarda la clave en el llavero de Apple para no pedir contraseña tras reiniciar y prueba la conexión.

### Clave protegida por hardware con Touch ID (FIDO2)

```
ssh-keygen -t ed25519-sk -C "tu-correo@ejemplo.com"
```

#### ¿Qué hace este comando?

Crea una clave de seguridad física vinculada al sensor Touch ID del Mac (requiere confirmación biométrica en cada operación con GitHub).

## 1.6 Firma Criptográfica de Commits con SSH y GPG

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The title bar in the top-right corner says "ZSH". The terminal contains four lines of text: "git config --global gpg.format ssh", "git config --global user.signingkey ~/.ssh/id\_ed25519.pub", "git config --global commit.gpgsign true", and "git config --global tag.gpgsign true".

```
git config --global gpg.format ssh
git config --global user.signingkey ~/.ssh/id_ed25519.pub
git config --global commit.gpgsign true
git config --global tag.gpgsign true
```

### ¿Qué hace este comando?

Firma automáticamente todos los commits con tu clave SSH para que GitHub los identifique con la insignia verde **Verified**.

---

## Parte II: Flujo de Trabajo Esencial (Nivel Novato)

### 2.1 Creación y Clonación de Repositorios desde Terminal

```
mkdir -p ~/Developer/mi-proyecto-mac && cd ~/Developer/mi-proyecto-mac
git init
echo "# Proyecto macOS Apple Silicon" > README.md
git add README.md
git commit -m "docs: inicializar proyecto con README"
gh repo create mi-proyecto-mac --public --source=. --remote=origin --push
```

#### ¿Qué hace este comando?

Crea el directorio en `~/Developer`, inicia el repositorio Git local, genera el primer commit y publica el repositorio en GitHub vinculando el remoto en un solo paso.

```
git clone git@github.com:usuario/mi-proyecto.git
```

#### ¿Qué hace este comando?

Descarga la copia completa del repositorio remoto a tu Mac mediante SSH.

### 2.2 El Ciclo de Estados: Working Tree, Index y Commit en Zsh

```
git status -s
git diff
git diff --staged
```

#### ¿Qué hace este comando?

Inspecciona el estado de los archivos y compara diferencias antes y después de enviarlas al área de preparación (staging).

## 2.3 Staging Selectivo y Convención de Commits

```
git add -p main.swift
```

### ¿Qué hace este comando?

Abre el visor interactivo de bloques para seleccionar manualmente qué líneas de código enviar al staging.

```
git commit -m "feat(core): aceleración neuronal con Apple Neural Engine"
```

### ¿Qué hace este comando?

Confirma los cambios usando el formato estandarizado **Conventional Commits** (`feat:`, `fix:`, `docs:`, `perf:`, `refactor:`, `chore:`).

## 2.4 Sincronización con GitHub

```
git fetch origin
git pull --rebase origin main
git push -u origin main
```

### ¿Qué hace este comando?

Descarga las novedades de GitHub, sitúa tus commits locales por encima de los remotos y publica la rama con seguimiento automático.

## 2.5 Control de Archivos Ignorados en macOS

Crea `.gitignore` para herramientas de desarrollo de Apple:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The title bar in the top-right corner says "PLAINTEXT". The terminal displays a list of files and directories:

```
build/  
DerivedData/  
*.xcuserstate  
*.xcuserdatad  
.build/  
Pods/  
.env  
*.log
```

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The title bar in the top-right corner says "ZSH". The terminal displays the command:

```
git rm -r --cached build/
```

**¿Qué hace este comando?**

Remueve la carpeta `build/` de Git si fue subida por descuido sin borrar los archivos del disco.

## Parte III: Ramas, Fusiones y Estrategias Colaborativas (Nivel Intermedio)

---

### 3.1 Gestión Eficiente de Ramas con `git switch` y Zsh

```
git switch -c feature/procesamiento-metal
git push -u origin feature/procesamiento-metal
git switch main
git branch -d feature/procesamiento-metal
git push origin --delete feature/procesamiento-metal
```

#### ¿Qué hace este comando?

Crea una nueva rama de desarrollo, la sube a GitHub, vuelve a `main`, borra la rama local y la elimina en el servidor remoto.

### 3.2 Los Tres Métodos de Merge en GitHub

- **Create a Merge Commit:** Une las ramas creando un commit conmemorativo que conserva el historial íntegro.
- **Squash and Merge:** Reduce toda la rama a un único commit limpio y conciso en `main`.
- **Rebase and Merge:** Traslada los commits secuencialmente al extremo de `main`, logrando una historia 100% lineal.

### 3.3 Modelos de Flujo: GitHub Flow, Git Flow y Forking

```
gh repo fork swiftlang/swift --clone
cd swift
git fetch upstream
git switch main
git merge upstream/main
git push origin main
```

### ¿Qué hace este comando?

Crea una bifurcación (fork) en tu cuenta de GitHub, la clona a tu Mac y mantiene sincronizada tu copia con el repositorio original.

## 3.4 Operaciones de Reorganización: Stash, Cherry-Pick y Rebase Interactivo

```
git stash push -m "Optimizaciones de memoria en curso"
git stash pop
git cherry-pick 7e3b12a
git rebase -i HEAD~3
```

### ¿Qué hace este comando?

Guarda cambios en el búfer temporal, los restaura, aplica un commit de otra rama o reescribe interactivamente los últimos 3 commits.

## 3.5 El Salvavidas: Recuperación con Reflog

```
git reflog
git reset --hard "HEAD@{1}"
```

### ¿Qué hace este comando?

Inspecciona el historial cronológico de movimientos de `HEAD` en tu Mac y revierte cualquier pérdida accidental a su estado previo.

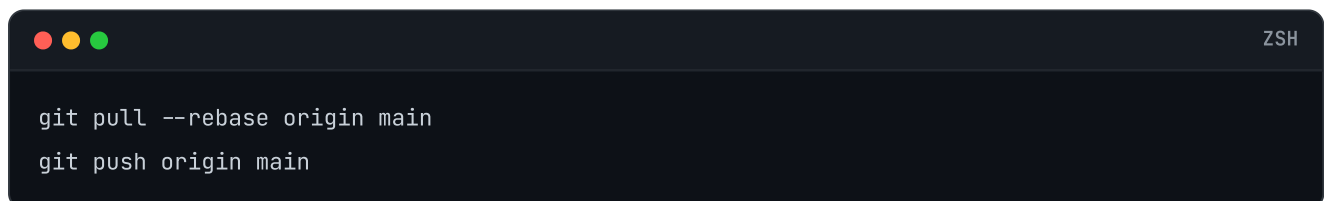
## Parte IV: Soluciones por Temas a la Edición Concurrente del Mismo Archivo

---

### 4.1 Tema 1: Prevención y Buenas Prácticas de Equipo

1. **Ramas independientes por tarea:** Prohibir commits directos sobre `main`. Trabajar siempre en ramas `feature/`.
  2. **Modularización:** Dividir archivos gigantes en estructuras más pequeñas para aislar responsabilidades.
  3. **Draft Pull Requests:** Publicar PRs en borrador para informar al equipo sobre qué archivos se están editando.
  4. **Sincronizaciones periódicas:** Ejecutar `git pull --rebase origin main` al comenzar y finalizar cada jornada.
- 

### 4.2 Tema 2: Fusión Automática en Distintas Líneas

A terminal window with a dark background and light text. The title bar shows three colored circles (red, yellow, green) on the left and 'ZSH' on the right. The terminal contains two lines of text: 'git pull --rebase origin main' and 'git push origin main'.

```
git pull --rebase origin main
git push origin main
```

#### *¿Qué hace este comando?*

*Integra el commit del compañero y coloca tus commits encima sin generar conflicto manual alguno.*

---

## 4.3 Tema 3: Conflicto Directo de Fusión en macOS

```
git config --global diff.tool vscode
git config --global difftool.vscode.cmd 'code --wait --diff $LOCAL $REMOTE'
git config --global merge.tool vscode
git config --global mergetool.vscode.cmd 'code --wait $MERGED'

git mergetool
git add src/App.swift
git rebase --continue
git push origin main
```

### ¿Qué hace este comando?

Abre automáticamente Visual Studio Code en macOS con el editor de 3 vías para resolver el conflicto visualmente, marca el archivo resuelto y concluye el rebase.

```
git rebase --abort
```

### ¿Qué hace este comando?

Cancela el proceso de rebase y devuelve el repositorio a su estado previo al conflicto.

## 4.4 Tema 4: Elección Total de Versión

```
# Conservar mi versión completa:
git checkout --ours src/App.swift
git add src/App.swift && git commit -m "resolve: mantener versión local"

# Aceptar la versión del compañero por completo:
git checkout --theirs src/App.swift
git add src/App.swift && git commit -m "resolve: aceptar versión remota"
```

### ¿Qué hace este comando?

Resuelve el conflicto escogiendo en su totalidad uno de los dos lados sin editar línea por línea.

## 4.5 Tema 5: Cambios Locales sin Confirmar al hacer Pull

```
git stash push -m "Cambios locales en curso"
git pull --rebase origin main
git stash pop
```

### ¿Qué hace este comando?

Pone a salvo tus modificaciones en la pila temporal, descarga las novedades de GitHub y reaplica tus cambios locales.

## 4.6 Tema 6: Push Rechazado por Desfase y Rebase Seguro

```
# NUNCA ejecutes git push --force
git pull --rebase origin main
git push origin main
```

### ¿Qué hace este comando?

Reubica tus commits locales por encima de los que tu compañero acaba de subir a GitHub y publica la rama sin sobrescribir nada.

## 4.7 Tema 7: Resolución de Conflictos en Pull Requests

```
gh pr checkout 22
git merge origin/main
code src/App.swift
git add src/App.swift
git commit -m "merge: reconciliar con rama main"
git push origin HEAD
```

### ¿Qué hace este comando?

Descarga la rama del PR con `gh`, incorpora `main`, resuelve conflictos en VS Code y actualiza el PR en GitHub.

## 4.8 Tema 8: Conflicto de Modificación vs Eliminación

```
# Para conservar el archivo:
git add src/App.swift && git commit -m "resolve: conservar archivo modificado"

# Para aceptar su eliminación:
git rm src/App.swift && git commit -m "resolve: confirmar eliminación del archivo"
```

### ¿Qué hace este comando?

Resuelve situaciones donde un usuario modificó el archivo y otro lo borró.

## 4.9 Tema 9: Archivos Binarios y Bloqueo con Git LFS en macOS

```
brew install git-lfs && git lfs install
git lfs track "*.mlmodel" --lockable
git lfs lock Models/VisionModel.mlmodel
git lfs locks
git lfs unlock Models/VisionModel.mlmodel
```

### ¿Qué hace este comando?

Configura Git LFS en macOS y bloquea modelos CoreML o archivos binarios en GitHub para prevenir modificaciones simultáneas.

# Parte V: Herramientas Modernas de Productividad Avanzada

---

## 5.1 Git Worktrees en macOS: Múltiples Ramas sin Conmutar

```
# Extraer una rama de hotfix en una carpeta paralela
git worktree add ../hotfix-audio hotfix/audio-engine-crash
cd ../hotfix-audio

# Compilar y commitear en paralelo sin alterar tu rama activa
git commit -am "fix(audio): corregir fuga de memoria en AVAudioEngine"
git push origin hotfix/audio-engine-crash

# Volver a la carpeta principal y remover el worktree
cd ../mi-proyecto-mac
git worktree list
git worktree remove ../hotfix-audio
```

### ¿Qué hace este comando?

Asocia una rama diferente a una carpeta física paralela sin tocar tu directorio de trabajo actual ni requerir `git stash`.

## 5.2 Depuración Binaria con Git Bisect y Rastreo con Blame

```
git bisect start
git bisect bad
git bisect good v1.0.0

# Tras probar cada compilación en macOS:
git bisect good # 0: git bisect bad

git bisect reset
```

### *¿Qué hace este comando?*

*Realiza una búsqueda binaria para encontrar automáticamente qué commit introdujo una regresión.*

```
git blame -L 20,40 src/MetalShader.metal
```

### *¿Qué hace este comando?*

*Muestra quién modificó cada línea del archivo, con el autor, fecha y hash de commit.*

## 5.3 GitHub Codespaces desde la Terminal de Mac

```
gh codespace create --repo usuario/mi-proyecto-mac --branch main
gh codespace list
gh codespace code -c nombre-del-codespace
```

### *¿Qué hace este comando?*

*Despliega un entorno de desarrollo completo en la nube de GitHub y lo abre en Visual Studio Code o en tu terminal.*

## 5.4 GitHub Copilot CLI en Zsh

```
gh extension install github/gh-copilot
gh copilot suggest "listar binarios universales en una carpeta en macOS"
gh copilot explain "lipo -create -output universal app_arm64 app_x86_64"
```

### *¿Qué hace este comando?*

*Agrega asistencia de IA generativa dentro de Zsh para redactar o interpretar comandos de terminal.*

## 5.5 Git Hooks y Validación con Pre-commit en macOS

```
brew install pre-commit

cat << 'EOF' > .pre-commit-config.yaml
repos:
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v4.6.0
    hooks:
      - id: check-added-large-files
        args: ['--maxkb=5000']
      - id: detect-private-key
      - id: end-of-file-fixer
EOF

pre-commit install
```

### ***¿Qué hace este comando?***

*Instala un gancho que valida automáticamente en cada commit local que no se suban archivos mayores a 5 MB ni claves privadas.*

## Parte VI: Gestión de Proyectos y Ecosistema GitHub

---

### 6.1 GitHub Issues, Hitos y Etiquetas desde Terminal

```
gh issue create --title "Fallo en compilación ARM64 nativa" --body "Error de enlace con librerías dinámicas." --label "bug,apple-silicon"
gh issue list --state open
gh issue close 24 --reason "completed"
```

#### *¿Qué hace este comando?*

Registra, consulta y cierra incidencias formales vinculadas al repositorio desde la terminal de macOS.

### 6.2 Pull Requests y Revisiones de Código con GitHub CLI

```
gh pr create --title "feat: aceleración neuronal en M3/M4" --body "Implementación con Apple Neural Engine."
gh pr checkout 9
gh pr review 9 --approve -b "Probado satisfactoriamente en M2 Max y M3 Pro."
```

#### *¿Qué hace este comando?*

Crea un Pull Request público, descarga la rama de un compañero a tu Mac y emite una aprobación oficial.

### 6.3 GitHub Projects (v2): Tableros y Automatización

```
gh project list
gh project item-add 2 --owner "equipo-apple" --url
"https://github.com/usuario/repo/issues/24"
```

### *¿Qué hace este comando?*

*Gestiona tableros Kanban interactivos y asocia tareas automáticamente.*

---

## 6.4 GitHub Discussions y Wikis Locales

ZSH

```
git clone git@github.com:usuario/mi-proyecto-mac.wiki.git
cd mi-proyecto-mac.wiki
echo "# Arquitectura Apple Silicon" > Arquitectura.md
git add Arquitectura.md
git commit -m "docs: documentar optimizaciones para SoC Apple"
git push origin master
```

### *¿Qué hace este comando?*

*Clona, edita y sincroniza la documentación Wiki del proyecto como un repositorio Git local.*

---

# Parte VII: Automatización y CI/CD con GitHub Actions

---

## 7.1 Estructura de Workflows y Sintaxis YAML

Los workflows se crean bajo `.github/workflows/*.yaml` y se ejecutan ante eventos como `push` o `pull_request`.

---

## 7.2 Pipelines con Runners Apple Silicon (`macos-14`, `macos-15`)

Crea `.github/workflows/macos-ci.yaml`:

```
name: macOS Apple Silicon Native CI

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  build-and-test:
    runs-on: macos-14 # Runner oficial con chip Apple Silicon (ARM64)

    steps:
      - name: Descargar repositorio
        uses: actions/checkout@v4

      - name: Seleccionar versión de Xcode
        run: sudo xcode-select -s /Applications/Xcode_15.4.app/Contents/Developer

      - name: Instalar dependencias mediante Homebrew
        run: |
          brew install cmake ninja

      - name: Compilar aplicación nativa Swift / C++
        run: |
          swift build -c release --triple arm64-apple-macosx

      - name: Ejecutar batería de pruebas
        run: |
          swift test

      - name: Subir binario compilado
        uses: actions/upload-artifact@v4
        with:
          name: app-macos-arm64
          path: .build/arm64-apple-macosx/release/
```

## 7.3 Compilación Cruzada, Binarios Universales y Caching

```
lipo -create -output BinarioUniversal Binario_arm64 Binario_x86_64
lipo -info BinarioUniversal
```

### ¿Qué hace este comando?

Crea un binario universal ejecutable tanto en procesadores Intel como en Apple Silicon.

## 7.4 Configuración de un Self-Hosted Runner ARM64 como Launchd Daemon

```
mkdir ~/actions-runner && cd ~/actions-runner
curl -o actions-runner-osx-arm64-2.316.0.tar.gz -L
https://github.com/actions/runner/releases/download/v2.316.0/actions-runner-osx-arm64-
2.316.0.tar.gz
tar xzf ./actions-runner-osx-arm64-2.316.0.tar.gz
./config.sh --url https://github.com/usuario/repo --token TU_TOKEN_DE_RUNNER
sudo ./svc.sh install
sudo ./svc.sh start
sudo ./svc.sh status
```

### ¿Qué hace este comando?

Convierte tu Mac mini o Mac Studio Apple Silicon en un ejecutor privado registrado como servicio continuo mediante `launchd`.

## Parte VIII: Distribución, Paquetes y Publicación

### 8.1 GitHub Releases: Binarios `.dmg`, `.pkg` y Arquitecturas ARM64

```
git tag -a v1.5.0 -m "release: versión 1.5.0 nativa Apple Silicon"
git push origin v1.5.0
gh release create v1.5.0 ./build/MiApp-1.5.0-arm64.dmg --title "MiApp v1.5.0 para macOS (Apple Silicon)" --generate-notes
```

#### ¿Qué hace este comando?

Publica formalmente una versión en GitHub, genera notas automáticas y adjunta el instalador `.dmg` para Apple Silicon.

### 8.2 GitHub Packages y Distribución mediante Homebrew Taps

```
# Formula/miapp.rb en tu repositorio homebrew-tap
class Miapp < Formula
  desc "Herramienta optimizada para Apple Silicon"
  homepage "https://github.com/usuario/miapp"
  url "https://github.com/usuario/miapp/releases/download/v1.5.0/miapp-1.5.0-arm64.tar.gz"
  sha256 "HASH_SHA256_DEL_ARCHIVO"
  version "1.5.0"

  def install
    bin.install "miapp"
  end
end
```

### 8.3 GitHub Pages: Sitios Estáticos y Documentación

Publica sitios de documentación generados por frameworks modernos (VitePress, Astro, Next.js) con despliegue automático mediante GitHub Actions.



# Parte IX: Seguridad, Gobernanza y Políticas de Repositorio

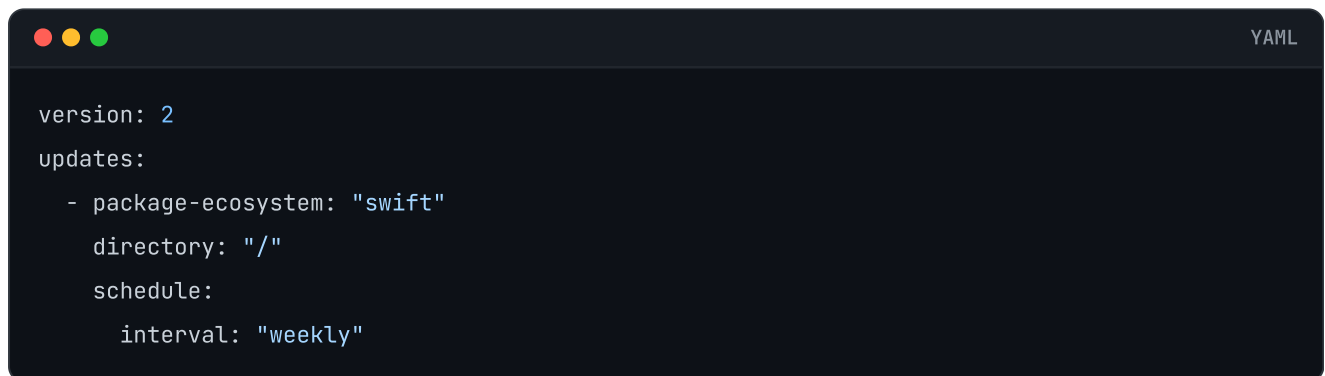
---

## 9.1 Branch Protection Rules y Rulesets

Desde **Settings** → **Rules** → **Rulesets**:

- Prohibir commits directos sin Pull Request.
- Exigir al menos 1 aprobación de código.
- Exigir que los pipelines de pruebas pasen satisfactoriamente.
- Bloquear force-pushes ( `git push --force` ).

## 9.2 Dependabot, Secret Scanning y Push Protection

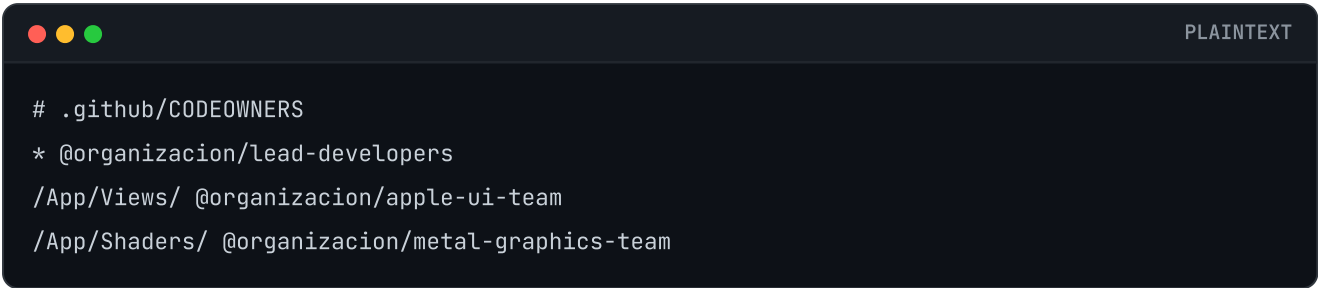


```
version: 2
updates:
  - package-ecosystem: "swift"
    directory: "/"
    schedule:
      interval: "weekly"
```

## 9.3 Análisis Estático con CodeQL

Inspecciona el código en cada PR para detectar fallos de concurrencia, accesos a punteros nulos o brechas de seguridad antes de fusionar.

## 9.4 Gobernanza con `CODEOWNERS` y Permisos de Equipo

A dark-themed terminal window with a title bar containing three colored circles (red, yellow, green) on the left and the word "PLAINTEXT" on the right. The terminal displays the content of a .github/CODEOWNERS file.

```
# .github/CODEOWNERS
* @organizacion/lead-developers
/App/Views/ @organizacion/apple-ui-team
/App/Shaders/ @organizacion/metal-graphics-team
```

# Parte X: Catálogo Maestro de Incidentes y Soluciones en macOS

Catálogo de incidentes críticos y su resolución en macOS Apple Silicon:

## 10.1 Incidente 1: Fuga Accidental de Secretos en macOS

**Escenario:** Se hizo commit de un archivo de configuración con API Keys o claves privadas `.p8` de Apple Developer.

```
brew install git-filter-repo
git filter-repo --path AuthKey.p8 --invert-paths --force
git push origin --force --all
git push origin --force --tags
```

*¿Qué hace este comando?*

*Reescribe la historia eliminando cualquier rastro del archivo secreto de la base de datos de Git.*

## 10.2 Incidente 2: Push Rechazado por Archivo Mayor a 100 MB

**Escenario:** GitHub rechaza el push por incluir un archivo `.dmg` o modelo de CoreML mayor a 100 MB.

```
# Si está en el commit más reciente:
git reset --soft HEAD~1
git rm --cached ModeloGigante.mlmodel
echo "ModeloGigante.mlmodel" >> .gitignore
git commit -m "chore: reconstruir commit excluyendo modelo pesado"
git push origin main
```

```
# Si quedó en commits intermedios antiguos:  
git filter-repo --strip-blobs-bigger-than 100M --force  
git push origin main
```

## 10.3 Incidente 3: Reversión Limpia de un Merge Roto en Producción

```
git log --oneline -n 5  
git revert -m 1 HASH_DEL_MERGE -m "revert: revertir merge defectuoso"  
git push origin main
```

### ¿Qué hace este comando?

Anula las modificaciones del merge preservando la integridad del historial para todos los clones.

## 10.4 Incidente 4: Rebase de una Rama Compartida

```
git fetch origin  
git switch rama-afectada  
git rebase --onto origin/rama-afectada @{upstream}
```

## 10.5 Incidente 5: Resurrección de una Rama Remota Borrada en GitHub

```
git reflog | grep "rama-borrada"  
git switch -c rama-borrada 7f6e5d4  
git push -u origin rama-borrada
```

## 10.6 Incidente 6: Corrección Masiva de Correo en Commits Históricos

```
git filter-repo --email-callback '
return email.replace(b"correo_invalido@mac.local", b"correo_oficial@empresa.com")
' --force
git push origin --force --all
```

## 10.7 Incidente 7: Ataques de Pwn Request en Runners de macOS

- Configura en **Settings** → **Actions** → **General**: **Require approval for all outside collaborators**.
- Nunca ejecute código de PRs externos con `pull_request_target` si el pipeline utiliza perfiles de aprovisionamiento de Apple o certificados de firma de código ( `Certificates.p12` ).

## 10.8 Incidente 8: Bucle Infinito de Workflows en GitHub Actions

Incluye `[skip ci]` en los mensajes de commit generados por bots automáticos en tus scripts:

```
- name: Formatear código Swift con SwiftFormat
  run: |
    git config user.name "github-actions[bot]"
    git config user.email "github-actions[bot]@users.noreply.github.com"
    git add .
    git diff-index --quiet HEAD || git commit -m "style: formateo automático de código [skip ci]"
    git push
```

## 10.9 Incidente 9: Optimización y Poda de Objetos Huérfanos en macOS

```
git count-objects -vH
git reflog expire --expire=now --all
git gc --prune=now --aggressive
```

***¿Qué hace este comando?***

*Empaqueta y optimiza los árboles de objetos de `.git`, eliminando archivos colgantes y reduciendo drásticamente el peso del repositorio en tu Mac.*

---

## 10.10 Incidente 10: Tags o Etiquetas Desincronizadas en macOS

ZSH

```
git tag -d $(git tag -l)
git fetch --tags --prune origin
```

***¿Qué hace este comando?***

*Limpiando la caché local de etiquetas y descarga los tags oficiales directamente desde GitHub.*